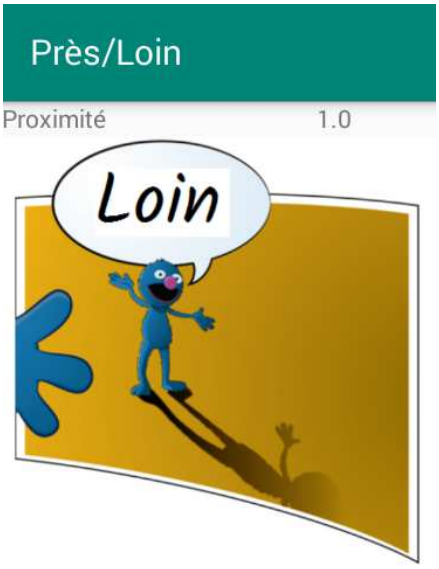
TP06 : Manipulation des Capteurs sous Android

Matière : ATELIER DEVELOPPEMENT MOBILE AVANCE	**Classes : SEM31**

Ce TP est rédigé avec des questions/réponses Gemini.



"**Pres/Loin**" est une application Android qui permet d'afficher la valeur interceptée par le capteur de proximité (1.0 : il n y a rien devant le capteur, 0.0 : il y a un obstacle devant le capteur) et de modifier l'image affichée suivant la valeur (pour 1.0 : l'id de l'image affichée est IMG_LOIN et pour 0.0 : l'id de l'image affichée est IMG_PRES).

Pour modifier l'image, utiliser

imgPL.setImageResource(IMG_PRES)

Voici l'énoncé détaillé et pas à pas de l'exercice **"Pres/Loin"**, structuré en 18 étapes pour concevoir l'application sous Android Studio de A à Z.

Étape 1 : Création du projet

Ouvrez **Android Studio** et lancez l'assistant de création d'un nouveau projet.

Étape 2 : Choix du modèle d'activité

Sélectionnez le modèle **Empty Views Activity** (ou *Empty Activity* selon la version) puis cliquez sur *Next*.

Étape 3 : Configuration du projet

Nommez votre application **Pres_Loin**, choisissez **Kotlin** comme langage de programmation, définissez une version minimale de SDK compatible, puis cliquez sur *Finish*.

Étape 4 : Activation du ViewBinding

Ouvrez le fichier de configuration Gradle du module (`build.gradle.kts` ou `build.gradle` du module `:app`). Dans le bloc `android { }`, activez la fonctionnalité de liaison de vues :

Kotlin

```
buildFeatures {
    viewBinding = true
}
```

Étape 5 : Synchronisation du projet

Cliquez sur le bouton **Sync Now** en haut à droite d'Android Studio pour appliquer l'activation du ViewBinding.

Étape 6 : Préparation des ressources graphiques

Importer les images d'illustration (représentant l'état près et l'état loin) au dossier des ressources drawables de votre projet (`app/res/drawable`),

Étape 7 : Conception de l'interface utilisateur (XML)

Importer le fichier de layout principal dans `app/res/layout`

Étape 8 : Configuration de la classe MainActivity

Ouvrez le fichier `MainActivity.kt` et faites en sorte que la classe implémente l'interface d'écoute **`SensorEventListener`**.

Étape 9 : Ajout de l'ImageView

Déclarer deux constates IMG_PRES et IMG_LOIN

```
companion object {
    private const val IMG_LOIN = R.drawable.loin00
    private const val IMG_PRES = R.drawable.pres00
}
```

Étape 10 : Déclaration des variables globales

Déclarez au sein de la classe :

- Une variable de type ViewBinding nommée **`amb`** (`private lateinit var amb: ActivityMainBinding`).
- Un gestionnaire de capteurs (`sensorManager`).
- Un capteur de type proximité (`proximitySensor`).

Étape 11 : Création de la méthode d'initialisation `initApp()`

Créez une méthode privée nommée `fun initApp()` qui se chargera de regrouper toute la configuration initiale de l'écran et des capteurs.

Étape 12 : Initialisation du ViewBinding dans `initApp()`

À l'intérieur de `initApp()`, gonflez la vue avec `amb = ActivityMainBinding.inflate(layoutInflater)` et appliquez-la à l'activité via `setContentView(amb.root)`.

Étape 13 : Initialisation du SensorManager dans `initApp()`

Récupérez le service système des capteurs dans la même méthode : `sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager`.

Étape 14 : Récupération du capteur de proximité

Toujours dans `initApp()`, instanciez le capteur cible : `proximitySensor = sensorManager.getDefaultSensor(Sensor.TYPE_PROXIMITY)`.

Étape 15 : Appel de `initApp()` dans `onCreate()`

Remplacez le contenu de la méthode de cycle de vie `onCreate()` de l'activité pour qu'elle appelle simplement la fonction `initApp()`.

Étape 16 : Gestion de l'activation dans `onResume()`

Surchargez la méthode `onResume()` pour enregistrer l'écouteur du capteur de manière sécurisée (sans utiliser d'expression lambda, en vérifiant que le capteur n'est pas nul dans une structure conditionnelle classique) avec un délai `SensorManager.SENSOR_DELAY_NORMAL`.

Étape 17 : Gestion de la mise en veille dans `onPause()`

Surchargez la méthode `onPause()` pour désenregistrer l'écouteur (`sensorManager.unregisterListener(this)`) afin d'économiser la batterie lorsque l'application passe en arrière-plan.

Étape 18 : Traitement des données du capteur

Implémentez la méthode obligatoire `onSensorChanged(event: SensorEvent)` :

- Vérifiez qu'il s'agit du capteur de proximité (`event.sensor.type == Sensor.TYPE_PROXIMITY`).
- Récupérez la valeur mesurée (`val distance = event.values[0]`).
- Mettez à jour le texte du `TextView` (`amb.tvValeur.text`).
- Modifiez dynamiquement l'image affichée dans l'ImageView : `amb.imgPL.setImageResource(R.drawable.img_pres)` si la distance vaut `0.0f`, sinon `amb.imgPL.setImageResource(R.drawable.img_loin)`.

Étape 18 : Programmer le bouton suivant pour changer les images

```
companion object {
    // Tableau d'images pour l'état "Près"
    private val TAB_PRES = intArrayOf(
        R.drawable.pres00,
        R.drawable.pres01,
        R.drawable.pres02,
        R.drawable.pres03
    )

    // Tableau d'images pour l'état "Loin"
    private val TAB_LOIN = intArrayOf(
        R.drawable.loin00,
        R.drawable.loin01,
        R.drawable.loin02,
        R.drawable.loin03
    )
}
```

```
// Index de l'image courante dans le tableau
private var indexImage = 0
```